# Lab 11: Government Data APIs

INFO 4617/5617 · Week 11 · Chapter 11, Government Data APIs

This week every class is live coding: one lab in three parts, one a day. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

We'll use four public APIs, and each one shows an idea that holds for APIs in general:

- **Part 1, Monday:** the **Census Bureau**, and what can I ask for?;
- **Part 2, Wednesday: FRED**, the Federal Reserve Bank of St. Louis's data service, and let the server compute; then the **Energy Information Administration (EIA)**, and let the server filter;
- **Part 3, Friday: Congress.gov**, and follow the links; then a join of two of the APIs.

Along the way, short ⏱️ microsprints ask you to change one thing, alone or in pairs, and share what you find.

**Before class:** open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Request four free keys and set them as environment variables before you start Jupyter. The chapter's "Setting Environment Variables" shows two ways; the simpler for this course stores them with the `webdata` environment, one line per key in a terminal (on Windows, the Anaconda Prompt): `conda env config vars set CENSUS_KEY="your-census-key" -n webdata`. The Missing Manual's Chapter 34 explains both.

- `CENSUS_KEY`, from the Census Bureau, for Monday; click the activation link in its email before you use it;
- `FRED_KEY`, from FRED, and `EIA_KEY`, from the EIA, for Wednesday;
- `DATAGOV_KEY`, from api.data.gov, for Friday's Congress.gov.

Each part reads only the keys it needs.

On Wednesday and Friday, run every cell above where we stopped (**Run → Run All Above Selected Cell**), or start at that day's setup step.

> **Instructor key.** All three parts were run live with `labs/tools/run_key.py`, the last time on 2026-10-26, and every one of their cells ran. They follow the textbook at commit `a9ecce1`: chapter 11 after its second revision (#290), the move of its public-interest callout into "Social History and Public Interest" (#291), and Congress.gov and the Census–EIA join (#292). Part 1 uses the 2020–2024 American Community Survey five-year estimates, so a rerun gives the same numbers. (The 2018–2022 estimates the first version of Part 1 used gave 72 places without an income estimate rather than 83, and Campo had none.) Part 2's requests have fixed end dates, but FRED and the EIA revise recent values: the EIA's 2025 figures are preliminary. Part 3's Congress.gov outputs change whenever Congress acts: H.R. 7730's latest action, and each member's bills and how far they got. Parts 2 and 3 use the chapter's code without its `timeout`, `raise_for_status()`, and helper functions; Step 14's given loop adds a one-second pause between pages. Step 18's `bills_on_page()` and the given `sponsored_bills()` are the lab's, in the three moves of `labs/AUTHORING.md`; the chapter writes the loop inline, for all ten members.

# Part 1 — Monday: the Census, and what can I ask for?

Every API answers only the questions it was built for, in its own words. Before we can ask the Census Bureau anything, we have to find out what it can tell us (its variables) and about what (its geographies). The Census API answers both questions itself, in catalogs we can request like any other data. Finding out what an API offers, from the API, is called discovery.

## Step 0 — Set up, and keep the key out of the notebook (5 minutes)

An API key identifies you, counts your requests, and is yours alone. A key typed into a notebook travels with the notebook: into a GitHub repository, a class submission, a classmate's laptop. So read the key from an environment variable, as the chapter's "API Key Management" section does. If you haven't set it yet, the chapter's "Setting Environment Variables" shows how: Jupyter sees only the variables set before it started, so set the key, then restart Jupyter. The chapter's setup cell prints whether each of its four keys is set; `os.environ["NAME"]` goes a step further in one line, since a missing key raises a `KeyError` that names it. (The chapter also shows a second way, a `.gitignore`d `api_keys.json` file.) Today we need only `CENSUS_KEY`. Define the User-Agent too: these APIs ask for a key, but every request still says who sent it, in the book's form.

```
# Import os, requests, pandas (as pd), and matplotlib.pyplot (as plt)
# Define HEADERS, whose "User-Agent" is "WebDataScience/1.0 (your-email@colorado.edu)", with your email
# Read CENSUS_KEY into census_key with os.environ[...]; never print the key
import os
import requests
import pandas as pd
import matplotlib.pyplot as plt

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
census_key = os.environ["CENSUS_KEY"]
print("Census key found")
```

```
Census key found
```

### 💬 Discuss

- In week 10 we told Wikipedia who we are with a User-Agent. The Census Bureau also asks for a key: what can a key do that a User-Agent can't?
- Where could a key typed into this notebook end up: a commit, a screenshot, a shared notebook? The Census Bureau takes the key in the URL, so `response.url` contains it too. Which of those could you take back?
- If `CENSUS_KEY` isn't set, what does this cell do? Why is that better than finding out at the first request?

## Step 1 — What can I ask about? The variable catalog (7 minutes)

The American Community Survey's thousands of variables come in groups (the chapter's "What Can I Ask For? Variables" searches all 28,475 of them, about 10 MB; we'll ask for one group's catalog, which is much smaller). Group `B25034` is "Year Structure Built", and the group's catalog is itself an API: send it a GET with `requests.get()` and our `HEADERS`, and read the answer with `.json()`. Print only the estimates, the codes ending in `E`: the chapter skips the margins of error (`M`), but the group also holds annotation variables (`EA`, `MA`) that would triple the list. Browse all the groups at api.census.gov/data/2024/acs/acs5/groups.html.

```
# Request "https://api.census.gov/data/2024/acs/acs5/groups/B25034.json" with HEADERS; read the JSON into variables
# Loop over sorted(variables["variables"].items()), and print each code that ends in "E" with its label
variables_url = "https://api.census.gov/data/2024/acs/acs5/groups/B25034.json"
variables = requests.get(variables_url, headers=HEADERS).json()

for var_id, var_info in sorted(variables["variables"].items()):
    if var_id.endswith("E"):
        print(f"{var_id}: {var_info['label']}")
```

```
B25034_001E: Estimate!!Total:
B25034_002E: Estimate!!Total:!!Built 2020 or later
B25034_003E: Estimate!!Total:!!Built 2010 to 2019
B25034_004E: Estimate!!Total:!!Built 2000 to 2009
B25034_005E: Estimate!!Total:!!Built 1990 to 1999
B25034_006E: Estimate!!Total:!!Built 1980 to 1989
B25034_007E: Estimate!!Total:!!Built 1970 to 1979
B25034_008E: Estimate!!Total:!!Built 1960 to 1969
B25034_009E: Estimate!!Total:!!Built 1950 to 1959
B25034_010E: Estimate!!Total:!!Built 1940 to 1949
B25034_011E: Estimate!!Total:!!Built 1939 or earlier
NAME: Geographic Area Name
```

### 💬 Discuss

- What do the parts of `B25034_002E` tell you: the table, the line, and the final letter? Why would an estimate need an annotation variable beside it?
- In week 6, The Numbers hid its table among decoys. How is picking a variable from this catalog like that, and how is it different?
- Every cell today assumes its request works. If the Census Bureau answered a 429, or a web page with status 200, which line would fail, and would week 10's `raise_for_status()` have caught either?

## Step 2 — Where can I ask about? The geography catalog (6 minutes)

The variables are half of a question; the other half is a place. The Census divides the country into nested geographies (states, counties, places, tracts, and more), and the dataset's geography catalog lists every level it can answer for, and what each one requires: the larger geography you must name with it. It is an API too, at `geography.json`. Print the requirements of four levels, by their summary-level codes, as the chapter's "What Can I Ask For? Geographies" does: a state is `040`, a county `050`, a census tract `140`, and a place, such as a city, `160`.

```
# Request "https://api.census.gov/data/2024/acs/acs5/geography.json" with HEADERS; levels: the
↪  JSON's "fips" list
# Print the "name" and .get("requires") of each level whose "geoLevelDisplay" is "040", "050",
↪  "140", or "160"
geography_url = "https://api.census.gov/data/2024/acs/acs5/geography.json"
levels = requests.get(geography_url, headers=HEADERS).json()["fips"]

for level in levels:
    if level["geoLevelDisplay"] in ("040", "050", "140", "160"):
        print(f"{level['name']}: requires {level.get('requires')}")
```

```
state: requires None
county: requires ['state']
tract: requires ['state', 'county']
place: requires ['state']
```

### 💬 Discuss

- A city is a `place`. What else must a request about a place name, and why would a `tract` need a county as well as a state?
- Steps 1 and 2 asked the API what it knows before asking it anything. Without the two catalogs, how would you have found `B25034` or `place`?
- Which catalog would you check again next year, when the Census publishes the 2023 estimates, and why might its answer change?

## Step 3 — Ask about one place (5 minutes)

Step 2 says a place needs its state. Places are named by FIPS codes: Colorado is state `08`, and Boulder is place `07850`. As in the chapter's "Querying for a Place", ask for the place's name and the first few `B25034` estimates, with `get`, `for`, and `in` parameters, and the key. The answer is a list of lists whose first row is the header, and every value is a string, so make a DataFrame and turn the numbers into numbers with `pd.to_numeric`. Keep the name `census_base` (the chapter's): we use it again.

```
# census_base = "https://api.census.gov/data/2024/acs/acs5"
# params: get="NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E,B25034_005E",
 ↪  for="place:07850", in="state:08", key=census_key
# Request census_base with params and HEADERS; make df from rows 1 onward, with row 0 as the
 ↪  columns
# numeric_cols: the columns that start with "B25034"; convert them with pd.to_numeric, and show
 ↪  df
census_base = "https://api.census.gov/data/2024/acs/acs5"
params = {"get": "NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E,B25034_005E",
          "for": "place:07850", "in": "state:08", "key": census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()

df = pd.DataFrame(data[1:], columns=data[0])
numeric_cols = [c for c in df.columns if c.startswith("B25034")]
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric)
df
```

```
                    NAME  B25034_001E  B25034_002E  B25034_003E  B25034_004E  \
0  Boulder city, Colorado        47465          646         3836         3968

   B25034_005E state  place
0         5561    08  07850
```

### 💬 Discuss

- In week 5, `urlparse()` took apart a Census URL just like this request's. Which parameters name the variables, which the place, and which must never appear in a printed error?
- In week 7, the Wayback Machine's CDX API sent the same shape: a list of lists with a header row. Why might an API send data this way rather than as nested dictionaries?
- Why do the numbers arrive as strings, and why must a FIPS code like `08` stay one?

## Step 4 — Compare two places (13 minutes)

Several places can go in one request: `place:07850,45970` is Boulder and Longmont, as in the chapter's "Comparing Places". Ask for all ten decades of group `B25034`, lines `002` (built 2020 or later) to `011` (1939 or earlier); a list comprehension with an f-string writes the ten codes. Then draw the chapter's figure 11.5: one pair of bars per decade, labeled with the words from Step 1's catalog, and turned upside down first, so that time runs left to right, from the oldest decade to the newest. `DataFrame.plot.bar()` draws a pair for each row.

```
# housing_vars: "B25034_002E" to "B25034_011E", as [f"B25034_{line:03d}E" for line in range(2,
 ↪  12)]
# Set params["get"] to "NAME," plus housing_vars joined with commas, and params["for"] to
 ↪  "place:07850,45970"; request again
# df_compare: as in Step 3, with housing_vars converted; show it
housing_vars = [f"B25034_{line:03d}E" for line in range(2, 12)]
params["get"] = "NAME," + ",".join(housing_vars)
params["for"] = "place:07850,45970"
data = requests.get(census_base, params=params, headers=HEADERS).json()
```

```
df_compare = pd.DataFrame(data[1:], columns=data[0])
df_compare[housing_vars] = df_compare[housing_vars].apply(pd.to_numeric)
df_compare
```

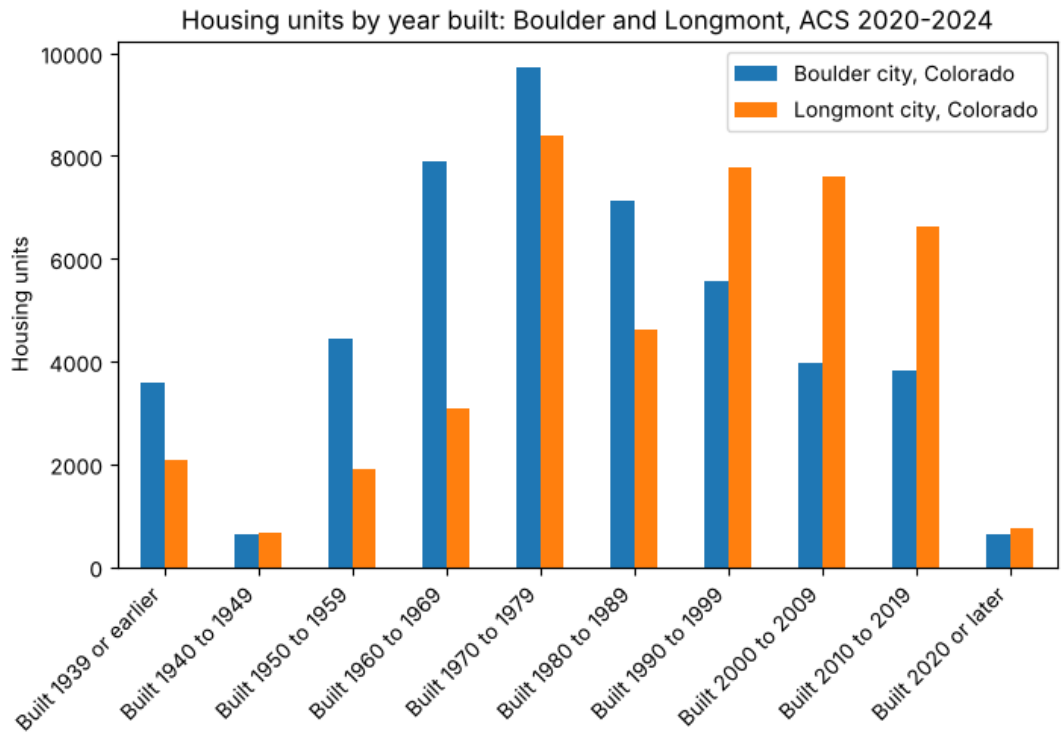
```
                     NAME  B25034_002E  B25034_003E  B25034_004E  \
0   Boulder city, Colorado          646         3836         3968
1  Longmont city, Colorado          761         6631         7597

   B25034_005E  B25034_006E  B25034_007E  B25034_008E  B25034_009E  \
0         5561         7141         9739         7904         4453
1         7781         4616         8402         3099         1899

   B25034_010E  B25034_011E state  place
0          641         3576    08  07850
1          671         2071    08  45970
```

```
# labels: each housing variable's label in Step 1's catalog, the part after its last "!!" (such
↪  as "Built 2010 to 2019")
# housing: df_compare indexed by NAME, its housing_vars renamed with labels, and transposed
# Plot housing.iloc[::-1] (oldest decade first) as bars; a title, a y label, no x label, a legend
↪  with no title
labels = {var: variables["variables"][var]["label"].split("!!")[-1] for var in housing_vars}
housing = df_compare.set_index("NAME")[housing_vars].rename(columns=labels).T

ax = housing.iloc[::-1].plot.bar(figsize=(8, 4.5))
ax.set(title="Housing units by year built: Boulder and Longmont, ACS 2020-2024", ylabel="Housing
↪  units", xlabel="")
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha="right")
ax.legend(title="")
plt.show()
```



### ⏱️ Microsprint (6 minutes, alone or in pairs)

A short version of Additional Exercise 1. Cities differ in size, so raw counts don't compare fairly; shares do.

**Do:** ask for the total (`B25034_001E`) and the three decades since 2000 (`_002E` to `_004E`) for five Colorado places, and compute the share of each city's housing built in 2000 or later. The key uses Boulder, Longmont, Fort Collins (`27425`), Lakewood (`43000`), and Castle Rock (`12415`); look up others in the FIPS code lists. **Share:** the city with the largest share, the one with the smallest, and the two shares.

```
# Microsprint: set params["get"] to NAME and B25034_001E to B25034_004E, params["for"] to five
 ↪  places, and request
# df_five: indexed by NAME, the four counts converted with pd.to_numeric
# since_2000: 002E + 003E + 004E; share: since_2000 / 001E; show share, largest first
params["get"] = "NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E"
params["for"] = "place:07850,45970,27425,43000,12415"
data = requests.get(census_base, params=params, headers=HEADERS).json()
df_five = pd.DataFrame(data[1:], columns=data[0]).set_index("NAME")
df_five = df_five[["B25034_001E", "B25034_002E", "B25034_003E",
 ↪  "B25034_004E"]].apply(pd.to_numeric)
since_2000 = df_five["B25034_002E"] + df_five["B25034_003E"] + df_five["B25034_004E"]
share = since_2000 / df_five["B25034_001E"]
share.sort_values(ascending=False).round(3)
```

```
NAME
Castle Rock town, Colorado     0.712
Fort Collins city, Colorado    0.365
Longmont city, Colorado        0.344
Lakewood city, Colorado        0.182
Boulder city, Colorado         0.178
dtype: float64
```

### 💬 Discuss

- Which of your five cities built the most of its housing since 2000, and which the least? What might explain the difference?
- Boulder's biggest decade was the 1970s, and Longmont has more units than Boulder from every decade since the 1990s. The chapter names one likely reason, Boulder's 1976 cap on building permits, the Danish Plan. What would you need to know to say it was the reason?
- The chart counts the units standing in 2020–2024, by the year they were built. What does it leave out, and what does a five-year estimate mean for a count labeled "Built 2020 or later"?

## Step 5 — Sanity-check a number (4 minutes)

A variable with the right-sounding label can still be the wrong one. The chapter's "Querying for a Place" checks its answer against a value it can judge, the estimate on data.census.gov's table. Do the same before you trust a pipeline. Ask for Boulder's median household income, `B19013_001E`. Does it look like a city's median income: tens of thousands of dollars, not 7 or 70 million?

```
# params: get="NAME,B19013_001E", for="place:07850", in="state:08", key=census_key; request it
# Print the city and its median household income, with a dollar sign and commas
params = {"get": "NAME,B19013_001E", "for": "place:07850", "in": "state:08", "key": census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()
income = int(data[1][1])
print(f"{data[1][0]}: median household income ${income:,}")
```

```
Boulder city, Colorado: median household income $87,493
```

### 💬 Discuss

- Does Boulder's median income look right? What did you compare it with?
- Every estimate has a margin of error, the variable with the same code ending in `M`: the chapter's Boulder County figure is $103,994, ±$2,268. What does the margin tell you that the estimate alone doesn't?
- Which checks like this one could a pipeline run on its own, every time it runs?

## Step 6 — When there's no estimate (9 minutes)

A wildcard asks for every place of a kind: `for=place:*` with `in=state:08` returns all of Colorado's cities, towns, and census-designated places, as in the chapter's "Every Place in a State, and a Number That Isn't". Ask for each one's median household income, and `describe()` the column before you trust it: look at its minimum and its maximum.

Eighty-three of the 482 places have no estimate in the 2020–2024 survey (72 did in 2018–2022): too few households answered for the Census Bureau to compute one. The API doesn't leave the value out. It sends a number that can't be an income, `-666666666`, and `-222222222` for its margin of error; the Bureau's notes on estimate and annotation values list what each code means. Then look closely at four places: ask for the population (`B01003_001E`), the median income, and its margin for Boulder and three small towns, Bonanza (`07571`), Campo (`11645`), and Carbonate (`12030`), and set every negative value aside with `mask()`, which turns it into a missing value, `NaN`.

```
# params: get="NAME,B19013_001E", for="place:*", in="state:08", key=census_key; request it
# places: a DataFrame of the rows; places["income"]: B19013_001E converted with pd.to_numeric
# Print how many places came back, and describe() the income column
params = {"get": "NAME,B19013_001E", "for": "place:*", "in": "state:08", "key": census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()
places = pd.DataFrame(data[1:], columns=data[0])
places["income"] = pd.to_numeric(places["B19013_001E"])
print(len(places), "places")
places["income"].describe().round()
```

```
482 places
```

```
count          482.0
mean    -114723934.0
std      251998296.0
min     -666666666.0
25%          43678.0
50%          72168.0
75%         106214.0
max         250001.0
Name: income, dtype: float64
```

```
# params: get="NAME,B01003_001E,B19013_001E,B19013_001M", for="place:07850,07571,11645,12030",
 ↪  in="state:08", key=census_key
# Request it; towns: indexed by NAME, the three columns converted with pd.to_numeric; show towns
params = {"get": "NAME,B01003_001E,B19013_001E,B19013_001M",
          "for": "place:07850,07571,11645,12030", "in": "state:08", "key": census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()
towns = pd.DataFrame(data[1:], columns=data[0]).set_index("NAME")
towns = towns[["B01003_001E", "B19013_001E", "B19013_001M"]].apply(pd.to_numeric)
towns
```

| NAME | B01003_001E | B19013_001E | B19013_001M |
|---|---|---|---|
| Bonanza town, Colorado | 20 | -666666666 | -222222222 |
| Boulder city, Colorado | 106433 | 87493 | 6221 |
| Campo town, Colorado | 58 | 53125 | 40478 |

```
Carbonate town, Colorado            0   -666666666   -222222222
```

```
# Print the mean of the median-income column as it is; then mask every value below zero, and
↪  print the mean again
print(f"Mean with the codes in: {towns['B19013_001E'].mean():,.0f}")
towns = towns.mask(towns < 0)
print(f"Mean with them masked: {towns['B19013_001E'].mean():,.0f}")
towns
```

```
Mean with the codes in: -333,298,178
Mean with them masked: 70,309
```

| NAME | B01003_001E | B19013_001E | B19013_001M |
|---|---|---|---|
| Bonanza town, Colorado | 20 | NaN | NaN |
| Boulder city, Colorado | 106433 | 87493.0 | 6221.0 |
| Campo town, Colorado | 58 | 53125.0 | 40478.0 |
| Carbonate town, Colorado | 0 | NaN | NaN |

**💬 Discuss**

- `describe()` showed both ends of the scale: `-666666666`, and a largest median of 250,001. One is a code for no estimate, and one is a cap: "$250,000 or more". What would each do to a ranking of places by income, and would you have noticed?
- Why might an API send a code like this rather than leave the value empty? What does a missing value lose that the code keeps?
- Campo has an estimate this time, with a margin of error almost as large. Is that more useful than Bonanza's code, or less?
- Carbonate has no residents, yet it has a row. What does the Census count as a place, and what does that mean for a table of "every place"?

### End of Part 1

On Wednesday, Part 2 asks FRED to compute for us, then the EIA to filter for us; on Friday, Part 3 follows Congress.gov's links, and joins two of the week's APIs. Each part starts with a setup step that imports and reads its keys again, so it doesn't need the earlier parts' cells to have run.

## Part 2 — Wednesday: FRED, and let the server compute; the EIA, and let the server filter

Today's two APIs both do work before they answer. FRED (Federal Reserve Economic Data), from the Federal Reserve Bank of St. Louis, will transform a time series for us: into changes, percent changes, or yearly averages. The EIA (U.S. Energy Information Administration) will cut its enormous tables down to the rows and columns we name. Asking the server to do that work, instead of downloading everything and doing it in pandas, is today's idea.

### Step 7 — Set up again (2 minutes)

Part 2 starts fresh, so a student who missed Monday can start here. Import what we need, define the User-Agent, and read two keys from the environment: `FRED_KEY` and `EIA_KEY`, as the chapter's "Reading the Keys in Python" does. `os.environ["NAME"]` stops with a `KeyError` that names a key you haven't set.

```
# Import os, requests, pandas (as pd), and matplotlib.pyplot (as plt); define HEADERS as in Step
↪  0, with your email
# Read FRED_KEY and EIA_KEY into fred_key and eia_key with os.environ[...]; never print a key
import os
```

```
import requests
import pandas as pd
import matplotlib.pyplot as plt

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
fred_key = os.environ["FRED_KEY"]
eia_key = os.environ["EIA_KEY"]
print("Both keys found")
```

```
Both keys found
```

**💬 Discuss**

- Monday's key went in a URL parameter called `key`. FRED's is `api_key`, and so is the EIA's. Why might every API name the same thing differently?
- If you print `response.url` for any request today, what will it show that it shouldn't?

## Step 8 — A FRED series, and what it is (6 minutes)

FRED (Federal Reserve Economic Data) names each of its 800,000 series with an ID, which you find on the series' web page (the chapter's tour, figure 11.2). Before downloading a series, ask the API's `series` endpoint what it is. Then ask the `series/observations` endpoint for its values. FRED sends every value as a string and writes a missing one as `"."`, so convert them with `pd.to_numeric(errors="coerce")`, which turns `"."` into `NaN`. The series is Boulder County's median household income, as in the chapter's "Finding Series IDs".

```
# fred_base = "https://api.stlouisfed.org/fred"; params: series_id="MHICO08013A052NCEN",
 ↪  api_key=fred_key, file_type="json"
# Request f"{fred_base}/series" with params and HEADERS; info: the JSON's "seriess"[0]; print its
 ↪  title, frequency, and units
fred_base = "https://api.stlouisfed.org/fred"
params = {"series_id": "MHICO08013A052NCEN", "api_key": fred_key, "file_type": "json"}
info = requests.get(f"{fred_base}/series", params=params, headers=HEADERS).json()["seriess"][0]
print(info["title"], "|", info["frequency"], "|", info["units"])
```

```
Estimate of Median Household Income for Boulder County, CO | Annual | Dollars
```

```
# Add observation_end="2024-12-31" to params; request f"{fred_base}/series/observations"
# income: a DataFrame of the JSON's "observations", date and value only; convert both; show the
 ↪  last 3 rows
params["observation_end"] = "2024-12-31"
data = requests.get(f"{fred_base}/series/observations", params=params, headers=HEADERS).json()
income = pd.DataFrame(data["observations"])[["date", "value"]]
income["date"] = pd.to_datetime(income["date"])
income["value"] = pd.to_numeric(income["value"], errors="coerce")
income.tail(3)
```

```
         date     value
33 2022-01-01   96584.0
34 2023-01-01   97170.0
35 2024-01-01  103293.0
```

**💬 Discuss**

- Monday's Census API answered with a list of lists. What shape is FRED's answer, and which is easier to turn into a DataFrame?
- The Census Bureau's estimate for Boulder County in 2020–2024 was $103,994; FRED's for 2024 is $103,293. The chapter says they come from different programs. Which would you cite,

and how would you say which it is?
- Why fix `observation_end`, when the series only goes to 2024 anyway?

## Step 9 — Let the server compute: `units` (8 minutes)

We'll ask FRED for observations several more times today, each time with one parameter changed. So gather Step 8's lines into one function, `fred_observations()`, the chapter's name: whatever changes from call to call is an argument, and `**params` passes along any other parameters. Then ask for the same series twice more, with `units` set to `"chg"`, the change in dollars from the year before, and to `"pch"`, the percent change, as the chapter's "Let the Server Compute: `units`" does. One series, three answers, and FRED computed two of them.

```
# Define fred_observations(series_id, **params): Step 8's observation lines, gathered
#   Add series_id, api_key, and file_type to params; request; build and convert the DataFrame;
 ↪  return it
def fred_observations(series_id, **params):
    params.update(series_id=series_id, api_key=fred_key, file_type="json")
    data = requests.get(f"{fred_base}/series/observations", params=params,
 ↪  headers=HEADERS).json()
    df = pd.DataFrame(data["observations"])[["date", "value"]]
    df["date"] = pd.to_datetime(df["date"])
    df["value"] = pd.to_numeric(df["value"], errors="coerce")
    return df
```

```
# change: fred_observations() with units="chg"; growth: with units="pch"; both with
 ↪  observation_end="2024-12-31"
# views: income's date's year and value, change's value, and growth's value as four columns; show
 ↪  the last 3 rows
change = fred_observations("MHICO08013A052NCEN", units="chg", observation_end="2024-12-31")
growth = fred_observations("MHICO08013A052NCEN", units="pch", observation_end="2024-12-31")
views = pd.DataFrame({"year": income["date"].dt.year, "dollars": income["value"],
                      "change": change["value"], "percent change": growth["value"]})
views.tail(3).round(1)
```

```
    year   dollars  change  percent change
33  2022   96584.0  6257.0             6.9
34  2023   97170.0   586.0             0.6
35  2024  103293.0  6123.0             6.3
```

### ⏱️ Microsprint (4 minutes, alone or in pairs)

Check what the server computed against the dollars.

**Do:** find the years in which Boulder County's median household income fell: the rows of `views` whose `change` is below zero. **Share:** how many years, which ones, and the largest fall in dollars.

```
# Microsprint: falls, the rows of views whose "change" is below zero; show them
falls = views[views["change"] < 0]
falls.round(1)
```

```
    year  dollars  change  percent change
12  2001  57079.0 -1522.0            -2.6
13  2002  55810.0 -1269.0            -2.2
20  2009  63757.0 -3003.0            -4.5
21  2010  62215.0 -1542.0            -2.4
23  2012  66894.0 -1207.0            -1.8
31  2020  88289.0   -52.0            -0.1
```

💬 **Discuss**

- Did the falls cluster around any events you know? What would you check before saying they were caused by them?
- FRED computed `change` from the same dollars we have. When is it worth asking the server for something you could compute yourself?
- `fred_observations()` has no error handling: if FRED answered with a maintenance page, which line would fail, and what would its error say?

## Step 10 — Let the server compute: `frequency` and `aggregation_method` (10 minutes)

A series' frequency is how often it is measured, and FRED can lower it before it sends the data, as the chapter's "Let the Server Compute: `frequency` and `aggregation_method`" shows. Take one series through three requests: `CPIAUCNS`, the Consumer Price Index (CPI) for all urban consumers in the United States, which the Bureau of Labor Statistics (BLS) publishes every month. Ask for it as published; then with `frequency="a"` and `aggregation_method="avg"`, which turn each year's twelve months into their mean; then with `units="pc1"` as well, the percent change of each year's mean from the year before's, which is the year's inflation. FRED applies the frequency first, then the units.

```
# years: observation_start="2015-01-01", observation_end="2025-12-31"; monthly: CPIAUCNS as
 ↪  published
# annual: frequency="a", aggregation_method="avg"; inflation: the same, plus units="pc1"
# table: annual's year, annual's value, and inflation's value; show it
years = {"observation_start": "2015-01-01", "observation_end": "2025-12-31"}
monthly = fred_observations("CPIAUCNS", **years)
annual = fred_observations("CPIAUCNS", frequency="a", aggregation_method="avg", **years)
inflation = fred_observations("CPIAUCNS", frequency="a", aggregation_method="avg", units="pc1",
 ↪  **years)
table = pd.DataFrame({"year": annual["date"].dt.year, "mean index": annual["value"],
                      "inflation": inflation["value"]})
table.round(1)
```

```
    year  mean index  inflation
0   2015       237.0        0.1
1   2016       240.0        1.3
2   2017       245.1        2.1
3   2018       251.1        2.4
4   2019       255.7        1.8
5   2020       258.8        1.2
6   2021       271.0        4.7
7   2022       292.7        8.0
8   2023       304.7        4.1
9   2024       313.7        2.9
10  2025       321.9        2.6
```

⏱️ **Microsprint (4 minutes, alone or in pairs)**

Check the server's math, as the chapter does, on the same series.

**Do:** from `monthly`, compute each year's mean with `groupby()` on the year, then each year's percent change with `pct_change()` times 100. **Share:** your numbers for 2024 and 2025 beside FRED's, and why your 2015 is missing.

```
# Microsprint: by_hand: monthly's values grouped by the date's year, averaged
# check: by_hand and its pct_change() * 100 as two columns; show the last 3 rows
by_hand = monthly.groupby(monthly["date"].dt.year)["value"].mean()
```

```
check = pd.DataFrame({"mean index": by_hand, "inflation": by_hand.pct_change() * 100})
check.tail(3).round(1)
```

```
      mean index  inflation
date
2023       304.7        4.1
2024       313.7        2.9
2025       321.9        2.6
```

💬 **Discuss**

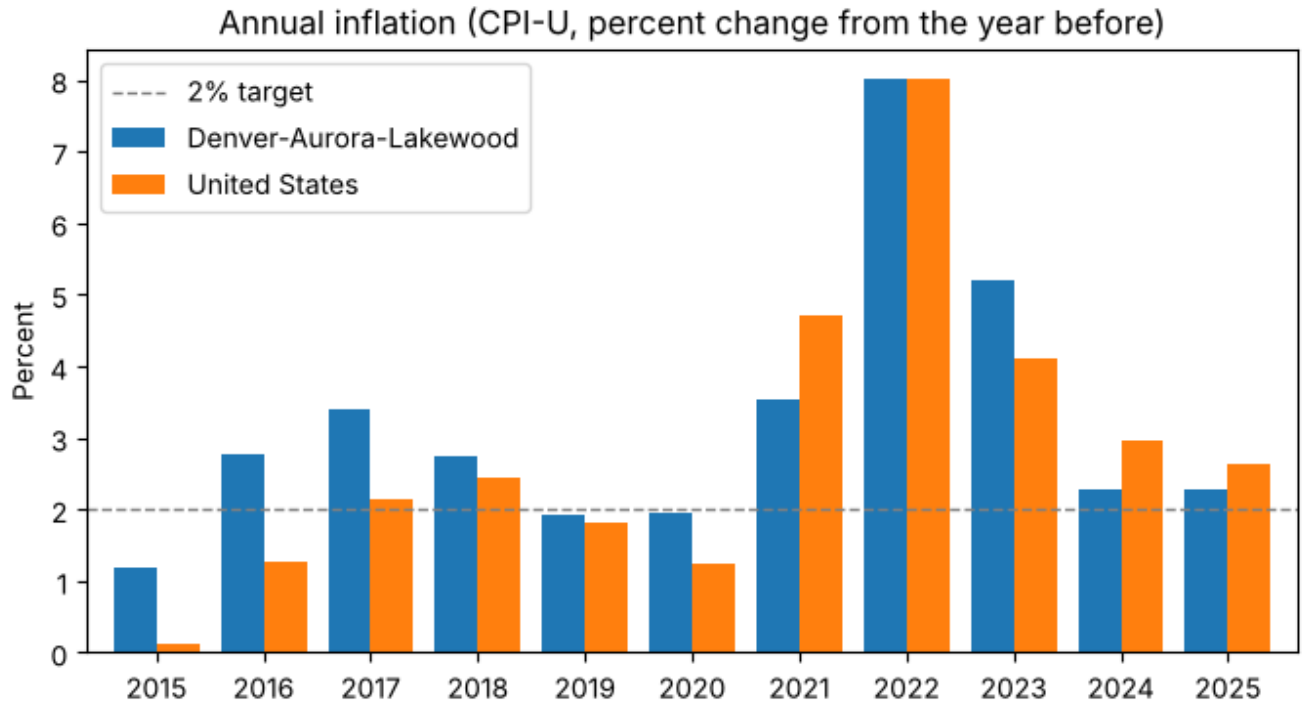
- Did your numbers match FRED's? If they hadn't, which would you trust, and how would you find out?
- Why can FRED give 2015's inflation when our pandas version can't?
- `aggregation_method` could also be `sum` or `eop`, the end of the period. Which makes sense for a price index, and which for something counted, like jobs added?

## Step 11 — Denver against the nation (5 minutes)

With one series understood, comparing two is asking each the same question. The BLS (Bureau of Labor Statistics) also publishes a CPI (Consumer Price Index) for the Denver area, which FRED carries as `CUUSA433SA0`. It is annual already, so it needs only `units="pc1"`. Merge it with Step 10's `inflation` on the date, and draw the chapter's paired bars: one pair a year, with a dashed line at 2 percent, the Federal Reserve's target, as in the chapter's figure 11.6. A DataFrame with one row per year and one column per place is what `DataFrame.plot.bar()` draws as pairs.

```
# denver: CUUSA433SA0 with units="pc1" and **years; cpi: denver merged with inflation on "date",
 ↪  suffixes ("_denver", "_national")
# pairs: cpi indexed by its date's year, the two value columns, renamed "Denver-Aurora-Lakewood"
 ↪  and "United States"
denver = fred_observations("CUUSA433SA0", units="pc1", **years)
cpi = denver.merge(inflation, on="date", suffixes=("_denver", "_national"))
pairs = cpi.set_index(cpi["date"].dt.year)[["value_denver", "value_national"]]
pairs.columns = ["Denver-Aurora-Lakewood", "United States"]
```

```
# Plot pairs as bars (rot=0); a dashed gray line at 2 labeled "2% target"; a title, a y label, no
 ↪  x label, and a legend
ax = pairs.plot.bar(figsize=(8, 4), rot=0, width=0.8)
ax.axhline(y=2, color="gray", linestyle="--", linewidth=1, label="2% target")
ax.set(title="Annual inflation (CPI-U, percent change from the year before)", ylabel="Percent",
 ↪  xlabel="")
ax.legend()
plt.show()
```




**💬 Discuss**

- In which years did Denver's prices rise faster than the nation's, and in which slower?
- Denver's series is annual and the nation's monthly. What would have gone wrong if we had merged the monthly series with the annual one, without `frequency="a"`?
- Housing is the largest part of the CPI. What would you ask FRED next to test whether rent explains the gap?

## Step 12 — EIA: routes that describe themselves (6 minutes)

The EIA's (U.S. Energy Information Administration's) API is a tree of **routes**, paths such as `electricity/retail-sales/` (documentation). Ask for a route without asking for its data, and it describes itself: at the top, the routes under it; further down, what its data can be filtered by and what it holds, as the chapter's "Routes That Describe Themselves" shows. Every answer is inside the JSON's `"response"`. It's Monday's idea, what can I ask for?, from a different API.

```
# eia_base = "https://api.eia.gov/v2"; request f"{eia_base}/electricity/" with params={"api_key":
 ↪  eia_key} and HEADERS
# Print each of the JSON's ["response"]["routes"]: its "id" and its "name"
eia_base = "https://api.eia.gov/v2"
electricity = requests.get(f"{eia_base}/electricity/", params={"api_key": eia_key},
 ↪  headers=HEADERS).json()["response"]
for child in electricity["routes"]:
    print(child["id"], "|", child["name"])
```

```
retail-sales | Electricity Sales to Ultimate Customers
electric-power-operational-data | Electric Power Operations (Annual and Monthly)
rto | Electric Power Operations (Daily and Hourly)
state-electricity-profiles | State Specific Data
operating-generator-capacity | Inventory of Operable Generators
facility-fuel | Electric Power Operations for Individual Power Plants (Annual and Monthly)
```

```
# meta: the "response" of f"{eia_base}/electricity/retail-sales/", requested the same way
# Print its "frequency" ids and its "facets" ids; then each column of its "data", with that
 ↪  column's "units"
```

```
meta = requests.get(f"{eia_base}/electricity/retail-sales/", params={"api_key": eia_key},
 ↪  headers=HEADERS).json()["response"]
print("Frequencies:", [f["id"] for f in meta["frequency"]])
print("Facets:", [f["id"] for f in meta["facets"]])
for column, about in meta["data"].items():
    print(column, "|", about["units"])
```

```
Frequencies: ['monthly', 'quarterly', 'annual']
Facets: ['stateid', 'sectorid']
revenue | million dollars
sales | million kilowatt hours
price | cents per kilowatt-hour
customers | number of customers
```

### 💬 Discuss

- Monday, the Census Bureau's catalogs were separate files, `variables.json` and `geography.json`. Here the route describes itself. Which would you rather explore, and why?
- The chapter's tour shows the EIA's API Dashboard. Where do its menus come from?
- What does a **facet** do that a data column doesn't?

## Step 13 — Let the server filter (8 minutes)

Add `data/` to the route, and it returns rows; each parameter narrows them, as the chapter's "Let the Server Filter" shows. `data[]` names the columns, `facets[stateid][]` and `facets[sectorid][]` keep only the values you list, `start` and `end` fix the years, and `sort` orders the rows. The square brackets are part of the parameters' names (documentation), and `requests` encodes them for you. Ask for the average price of electricity in Colorado's homes, every year from 2001 to 2025.

```
# params: frequency="annual", data[]="price", facets[stateid][]="CO", facets[sectorid][]="RES",
 ↪  start="2001", end="2025"
#   and sort[0][column]="period", sort[0][direction]="asc"; with api_key=eia_key
# Request f"{eia_base}/electricity/retail-sales/data/"; prices: its "response"; print its
 ↪  "total"; co_price: a DataFrame of its "data"
params = {"frequency": "annual", "data[]": "price", "facets[stateid][]": "CO",
 ↪  "facets[sectorid][]": "RES",
          "start": "2001", "end": "2025", "sort[0][column]": "period", "sort[0][direction]":
           ↪  "asc",
          "api_key": eia_key}
prices = requests.get(f"{eia_base}/electricity/retail-sales/data/", params=params,
 ↪  headers=HEADERS).json()["response"]
print("Rows:", prices["total"])
co_price = pd.DataFrame(prices["data"])
co_price[["period", "stateid", "sectorName", "price", "price-units"]].tail(3)
```

```
Rows: 25
```

| | period | stateid | sectorName | price | price-units |
|---|---|---|---|---|---|
| 22 | 2023 | CO | residential | 14.3 | cents per kilowatt-hour |
| 23 | 2024 | CO | residential | 14.92 | cents per kilowatt-hour |
| 24 | 2025 | CO | residential | 15.69 | cents per kilowatt-hour |

### ⏱️ Microsprint (4 minutes, alone or in pairs)

Change one facet.

**Do:** ask for the same price in another sector (`COM`, commercial, or `IND`, industrial), or for homes in another state (its two-letter code, such as `UT` or `NM`). The key asks for Colorado's commercial price.

**Share:** your sector or state, its 2025 price, and how it compares with Colorado's homes.

```
# Microsprint: set params["facets[sectorid][]"] (or params["facets[stateid][]"]) to your choice;
 ↪  request again
# Show the last 3 rows of period, stateid, sectorName, and price
params["facets[sectorid][]"] = "COM"
other = requests.get(f"{eia_base}/electricity/retail-sales/data/", params=params,
 ↪  headers=HEADERS).json()["response"]
pd.DataFrame(other["data"])[["period", "stateid", "sectorName", "price"]].tail(3)
```

```
   period stateid  sectorName  price
22   2023      CO  commercial  11.64
23   2024      CO  commercial  11.71
24   2025      CO  commercial  12.37
```

💬 **Discuss**

- How many rows came back, out of a route that holds every state and sector by month, quarter, and year? What did filtering on the server save?
- The prices arrive as strings. Which line would fail if you tried to average them?
- Why do businesses pay less per kilowatt-hour than homes?

## Step 14 — Paging with `offset` and `length` (7 minutes)

The EIA's (U.S. Energy Information Administration's) API returns at most 5,000 rows a request. Colorado's monthly generation from every source, 2001 to 2025, is more than that. The response says how many rows match in `total`, and you ask for the next page with `offset`, the rows to skip, and `length`, the rows to return (documentation; the chapter's "Paging with `offset` and `length`"). Ask for two pages by hand. Then read the chapter's loop, which is given, and use it. Compare it with week 10's: Wikipedia sent a `continue` token; the EIA tells you the total, and you keep count.

```
# params: frequency="monthly", data[]="generation", facets[location][]="CO",
 ↪  facets[sectorid][]="99", start="2001-01", end="2025-12",
#   sorted by "period" and then "fueltypeid", ascending, with api_key=eia_key, offset=0,
 ↪  length=5000
# gen_url: f"{eia_base}/electricity/electric-power-operational-data/data/"; request page one;
 ↪  print its "total" and its number of rows
params = {"frequency": "monthly", "data[]": "generation", "facets[location][]": "CO",
 ↪  "facets[sectorid][]": "99",
          "start": "2001-01", "end": "2025-12",
          "sort[0][column]": "period", "sort[0][direction]": "asc",
          "sort[1][column]": "fueltypeid", "sort[1][direction]": "asc",
          "api_key": eia_key, "offset": 0, "length": 5000}
gen_url = f"{eia_base}/electricity/electric-power-operational-data/data/"
page = requests.get(gen_url, params=params, headers=HEADERS).json()["response"]
print("Total:", page["total"], "| this page:", len(page["data"]))
```

```
Total: 9289 | this page: 5000
```

```
# Set params["offset"] to 5000; request page two; print its number of rows and its first row's
 ↪  "period"
params["offset"] = 5000
page = requests.get(gen_url, params=params, headers=HEADERS).json()["response"]
print("This page:", len(page["data"]), "| starts at", page["data"][0]["period"])
```

```
This page: 4289 | starts at 2015-04
```

```
# Given: the chapter's loop, which keeps asking until it has the total, a second apart
import time
```

```
def eia_get_all(route, params, page_size=5000):
    rows = []
    while True:
        page_params = dict(params, api_key=eia_key, offset=len(rows), length=page_size)
        page = requests.get(f"{eia_base}/{route}", params=page_params,
 ↪  headers=HEADERS).json()["response"]
        print(f"offset {len(rows):>5}: {len(page['data'])} rows of {page['total']}")
        rows.extend(page["data"])
        if len(rows) >= int(page["total"]) or not page["data"]:
            break
        time.sleep(1)
    return pd.DataFrame(rows)

monthly_gen = eia_get_all("electricity/electric-power-operational-data/data/", params)
print(len(monthly_gen), "rows;", monthly_gen["fueltypeid"].nunique(), "energy sources")
```

```
offset     0: 5000 rows of 9289
offset  5000: 4289 rows of 9289
9289 rows; 36 energy sources
```

**💬 Discuss**

- The loop stops when it has `total` rows. Why not stop when the response stops saying `"incomplete return"`, as the chapter explains?
- Why sort when you page? What could happen to a row between page one and page two if you didn't?
- Before writing a loop like this, what would you try first to make the answer fit in one page?

### End of Part 2

On Friday, Part 3 follows Congress.gov's links from one record to the next, joins the Census Bureau's data with the EIA's, and ends with what we learned. It starts with its own setup step.

## Part 3 — Friday: Congress.gov, and follow the links; then a join

A bill's record on Congress.gov, the Library of Congress's site for legislation, holds little data of its own. Mostly it holds links: to the bill's actions, its cosponsors, its text, each a record with its own address. Following those links, from one record to the records about it, is today's idea. Then we put two of the week's agencies in one table, and find that the hard part of a join is the identifier.

### Step 15 — Set up again (2 minutes)

Import what we need, define the User-Agent, and read three keys: `DATAGOV_KEY` for Congress.gov, and `CENSUS_KEY` and `EIA_KEY` for the join. Every api.data.gov service, Congress.gov's among them, accepts the key in a header, `X-Api-Key` (api.data.gov's developer manual), so make a copy of `HEADERS` that carries it, as the chapter's "A Bill and the Records It Links To" does. In a header, the key stays out of every URL we print.

```
# Import os, io, requests, pandas (as pd), and matplotlib.pyplot (as plt); define HEADERS as in
 ↪  Step 0, with your email
# Read DATAGOV_KEY, CENSUS_KEY, and EIA_KEY into datagov_key, census_key, and eia_key; never
 ↪  print a key
# congress_headers: a copy of HEADERS, with "X-Api-Key" set to datagov_key
import os
```

```
import io
import requests
import pandas as pd
import matplotlib.pyplot as plt

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
datagov_key = os.environ["DATAGOV_KEY"]
census_key = os.environ["CENSUS_KEY"]
eia_key = os.environ["EIA_KEY"]

congress_headers = HEADERS.copy()
congress_headers["X-Api-Key"] = datagov_key
print("All three keys found")
```

```
All three keys found
```

**💬 Discuss**

- Monday's key went in the URL, as `key`, and Wednesday's as `api_key`. Today's goes in a header. What can a header do that a URL parameter can't?
- One api.data.gov key works for Congress.gov and many other federal APIs. What does sharing one key across them mean for your rate limit?

## Step 16 — A bill and the records it links to (5 minutes)

Ask Congress.gov's API (documentation) for H.R. 7730, the bill in the chapter's tour, figure 11.4: `bill/119/hr/7730`, the 119th Congress's House bill 7730, with `format=json`. Print its title, its sponsor, and its latest action. Then look at the rest of the record: most of its fields are dictionaries holding a `count` and a `url`, the address of more records.

```
# congress_base = "https://api.congress.gov/v3"; request f"{congress_base}/bill/119/hr/7730" with
 ↪  params={"format": "json"} and congress_headers
# bill: the JSON's "bill"; print its "title", its first sponsor's "fullName", and its
 ↪  "latestAction"'s date and text
# Loop over bill.items(): for each value that is a dict with a "url", print the field, its
 ↪  "count", and its "url"
congress_base = "https://api.congress.gov/v3"
bill = requests.get(f"{congress_base}/bill/119/hr/7730", params={"format": "json"},
 ↪  headers=congress_headers).json()["bill"]
print(bill["title"])
print("Sponsor:", bill["sponsors"][0]["fullName"])
print("Latest action:", bill["latestAction"]["actionDate"], bill["latestAction"]["text"])

for field, value in bill.items():
    if isinstance(value, dict) and "url" in value:
        print(field, value.get("count"), value["url"])
```

```
Bankruptcy Threshold Adjustment Act
Sponsor: Rep. Cline, Ben [R-VA-6]
Latest action: 2026-10-14 Message on Senate action sent to the House.
actions 18 https://api.congress.gov/v3/bill/119/hr/7730/actions?format=json
committees 1 https://api.congress.gov/v3/bill/119/hr/7730/committees?format=json
cosponsors 6 https://api.congress.gov/v3/bill/119/hr/7730/cosponsors?format=json
relatedBills 2 https://api.congress.gov/v3/bill/119/hr/7730/relatedbills?format=json
subjects 4 https://api.congress.gov/v3/bill/119/hr/7730/subjects?format=json
summaries 1 https://api.congress.gov/v3/bill/119/hr/7730/summaries?format=json
textVersions 5 https://api.congress.gov/v3/bill/119/hr/7730/text?format=json
```

```
titles 7 https://api.congress.gov/v3/bill/119/hr/7730/titles?format=json
```

### 💬 Discuss

- Which of these links would you follow to find out whether the bill became law, and which to find out who supports it?
- Week 10's Wikipedia API answered `prop=revisions` inside one response. Why might Congress.gov send a link instead of the records themselves?

## Step 17 — Follow the links (7 minutes)

Each link is a complete request: the address, with `format=json` already on it. Send it with `congress_headers`, and the API answers with the records it promised, as in the chapter's "Following the Links". Follow two: the cosponsors, and the actions, the steps the bill has taken. The API lists actions newest first; print the milestones, the actions whose text starts with "Introduced in", "Passed/agreed to in", or "Became Public Law", oldest first. `str.startswith()` takes a tuple of beginnings. Some milestones are recorded twice, in the same words, so print each once.

```
# Request bill["cosponsors"]["url"] with congress_headers; print each of its "cosponsors":
 ↪  "sponsorshipDate" and "fullName"
cosponsors = requests.get(bill["cosponsors"]["url"],
 ↪  headers=congress_headers).json()["cosponsors"]
for person in cosponsors:
    print(person["sponsorshipDate"], person["fullName"])
```

```
2026-02-26 Rep. Correa, J. Luis [D-CA-46]
2026-02-26 Rep. Lee, Laurel M. [R-FL-15]
2026-02-26 Rep. Neguse, Joe [D-CO-2]
2026-03-24 Rep. Gooden, Lance [R-TX-5]
2026-03-25 Rep. Lofgren, Zoe [D-CA-18]
2026-06-08 Rep. Taylor, David J. [R-OH-2]
```

```
# actions: request bill["actions"]["url"] with params={"limit": 250} and congress_headers; take
 ↪  the JSON's "actions"
# milestones: the tuple ("Introduced in", "Passed/agreed to in", "Became Public Law"); shown: an
 ↪  empty set
# Loop over reversed(actions): print the date and the first 60 characters of each milestone's
 ↪  text not yet in shown; add it to shown
actions = requests.get(bill["actions"]["url"], params={"limit": 250},
 ↪  headers=congress_headers).json()["actions"]
print(len(actions), "actions")

milestones = ("Introduced in", "Passed/agreed to in", "Became Public Law")
shown = set()
for action in reversed(actions):
    if action["text"].startswith(milestones) and action["text"] not in shown:
        print(action["actionDate"], action["text"][:60])
        shown.add(action["text"])
```

```
18 actions
2026-02-26 Introduced in House
2026-09-16 Passed/agreed to in House: On motion to suspend the rules an
2026-09-28 Passed/agreed to in Senate: Passed Senate without amendment
```

### ⏱️ Microsprint (3 minutes, alone or in pairs)

Follow one more link.

**Do:** request `bill["textVersions"]["url"]`, and print each of its `textVersions`: its `type` and its `date`. The key does this one; `committees` or `subjects` work the same way. **Share:** how many versions of the text there are, and what the newest one is called.

```
# Microsprint: request bill["textVersions"]["url"] with congress_headers; print each
 ↪  "textVersions" item's "type" and "date"
versions = requests.get(bill["textVersions"]["url"],
 ↪  headers=congress_headers).json()["textVersions"]
for version in versions:
    print(version["type"], version["date"])
```

```
Enrolled Bill None
Received in Senate 2026-09-17T04:00:00Z
Engrossed in House 2026-09-16T04:00:00Z
Reported in House 2026-08-27T04:00:00Z
Introduced in House 2026-02-26T05:00:00Z
```

### 💬 Discuss

- Did we build any of these three addresses ourselves? What would have to change in our code if Congress.gov moved its actions to a new address?
- Boulder's representative, Joe Neguse, is one of the bill's original cosponsors. What does a cosponsorship tell you about a member, and what doesn't it?
- The newest text version has no date. What would that do to a chart of versions over time?

## Step 18 — Paging with `next` (10 minutes)

A member's `sponsored-legislation` lists everything they have sponsored, in every Congress, at most 250 items a page (documentation). The response's `pagination` holds `next`, the full address of the next page, and leaves it out on the last page. That's the week's third way to page: Wikipedia sent a `continue` token, the EIA an `offset` we computed, and Congress.gov sends the whole next request. Page through what Joe Neguse (`N000191`), Boulder's representative, has sponsored, as the chapter's "Paging with `next`: Colorado's Delegation" does for all ten members, in three moves: two pages by hand, one page as a function, then the loop, which is given. Keep the bills of the 119th Congress, House bills (`HR`) and Senate bills (`S`).

```
# page_one: request f"{congress_base}/member/N000191/sponsored-legislation?format=json&limit=250"
 ↪  with congress_headers; print its "pagination"
# page_two: request page_one["pagination"]["next"] the same way; print its "pagination", and its
 ↪  number of items
page_one =
 ↪  requests.get(f"{congress_base}/member/N000191/sponsored-legislation?format=json&limit=250",
 ↪  headers=congress_headers).json()
print(page_one["pagination"])
page_two = requests.get(page_one["pagination"]["next"], headers=congress_headers).json()
print(page_two["pagination"], len(page_two["sponsoredLegislation"]), "items")
```

```
{'count': 368, 'next': 'https://api.congress.gov/v3/member/N000191/sponsored-legislation?offset=
 ↪  250&limit=250&format=json'}
{'count': 368, 'prev': 'https://api.congress.gov/v3/member/N000191/sponsored-legislation?offset=
 ↪  0&limit=250&format=json'} 118
 ↪  items
```

```
# Define bills_on_page(url): request url; return a list of the 119th Congress's HR and S items
 ↪  (their "type", "number", "title",
#   and "latestAction" text), and the page's "pagination".get("next")
# rows, next_url = bills_on_page(...) for Neguse's first page; print len(rows) and next_url
def bills_on_page(url):
```

```
    """One page of sponsored legislation: its 119th-Congress bills, and the next page's address
     ↪  (None on the last page)."""
    page = requests.get(url, headers=congress_headers).json()
    rows = []
    for item in page["sponsoredLegislation"]:
        if item.get("congress") == 119 and item.get("type") in ("HR", "S"):
            rows.append({"type": item["type"], "number": item["number"], "title": item["title"],
                         "latest": item["latestAction"]["text"]})
    return rows, page["pagination"].get("next")

rows, next_url =
 ↪  bills_on_page(f"{congress_base}/member/N000191/sponsored-legislation?format=json&limit=250")
print(len(rows), "bills on page one; next:", next_url)
```

```
88 bills on page one; next: https://api.congress.gov/v3/member/N000191/sponsored-legislation?off
 ↪  set=250&limit=250&format=json
```

```
# Given: the loop, which follows "next" until there is none, and returns every 119th-Congress
 ↪  bill as a DataFrame
def sponsored_bills(bioguide_id):
    """Every House and Senate bill a member sponsored in the 119th Congress, following each
     ↪  page's "next"."""
    url = f"{congress_base}/member/{bioguide_id}/sponsored-legislation?format=json&limit=250"
    rows = []
    while url:
        page = requests.get(url, headers=congress_headers).json()
        for item in page["sponsoredLegislation"]:
            if item.get("congress") == 119 and item.get("type") in ("HR", "S"):
                rows.append({"type": item["type"], "number": item["number"], "title":
 ↪  item["title"],
                             "latest": item["latestAction"]["text"]})
        url = page["pagination"].get("next")
    return pd.DataFrame(rows)

bills = sponsored_bills("N000191")
print(len(bills), "bills in the 119th Congress")
bills.head(3)
```

```
88 bills in the 119th Congress
```

| | type | number | title | latest |
|---|---|---|---|---|
| 0 | HR | 10688 | Family Stability and Opportunity Vouchers Act ... | Referred to the House Committee on Financial S... |
| 1 | HR | 10584 | Wildfire and Grid Reliability Act | Referred to the House Committee on Energy and ... |
| 2 | HR | 8523 | Public Lands Workforce Stability Act | Referred to the Committee on Natural Resources... |

### ⏱️ Microsprint (4 minutes, alone or in pairs)

Another member, another count.

**Do:** call `sponsored_bills()` for another member of Colorado's delegation. IDs: Jeff Hurd, `H001100`; Brittany Pettersen, `P000620`; Gabe Evans, `E000300`; Michael Bennet, `B001267`; John Hickenlooper, `H000273`. The key uses Jeff Hurd. Then count the bills whose latest action starts with "Referred to", which

have gone no further than a committee; `str.startswith()` works on a whole column. **Share:** the member, their number of bills, and how many are still in committee.

```
# Microsprint: other = sponsored_bills("H001100") (or your member); stuck: its rows whose
 ↪  "latest" starts with "Referred to"
# Print len(other) and len(stuck)
other = sponsored_bills("H001100")
stuck = other[other["latest"].str.startswith("Referred to")]
print(len(other), "bills;", len(stuck), "with a latest action of a referral to committee")
```

```
27 bills; 15 with a latest action of a referral to committee
```

💬 **Discuss**

- A latest action of "Referred to" is a guess at how far a bill got. The chapter follows every bill's `actions` link instead. What does that cost, in requests and minutes, and what does it buy?
- Compare `bills_on_page()` with the given loop. What does the loop add, and how does it know when to stop?
- Wikipedia's `continue`, the EIA's `offset`, Congress.gov's `next`: which one asks you to keep count, and which one could you get wrong?

## Step 19 — Two agencies, one table (10 minutes)

What does electricity cost a household, against what households earn? The Census Bureau knows each state's median household income; the EIA knows what its homes paid for electricity. The hard part is the identifier: the Census Bureau names Colorado `08`, its FIPS code, and the EIA `CO`, its postal code. A **crosswalk** pairs the two, and the Census Bureau publishes one, listed on its FIPS codes page. Join through it, as the chapter's "Combining Census and EIA Data" does, with an outer `merge()` and `indicator=True`, so that the rows that didn't match are kept and labeled.

```
# states: the 2024 ACS's NAME and B19013_001E for "state:*" (with census_key), as a DataFrame;
 ↪  add states["income"], converted
# power: the EIA's electricity/retail-sales/data/ for 2024, annual, data[] revenue and customers,
 ↪  facets[sectorid][] RES, as a DataFrame
states_url = "https://api.census.gov/data/2024/acs/acs5"
data = requests.get(states_url, params={"get": "NAME,B19013_001E", "for": "state:*", "key":
 ↪  census_key}, headers=HEADERS).json()
states = pd.DataFrame(data[1:], columns=data[0])
states["income"] = pd.to_numeric(states["B19013_001E"])

params = {"frequency": "annual", "data[]": ["revenue", "customers"], "facets[sectorid][]": "RES",
          "start": "2024", "end": "2024", "api_key": eia_key}
power = pd.DataFrame(requests.get("https://api.eia.gov/v2/electricity/retail-sales/data/",
 ↪  params=params, headers=HEADERS).json()["response"]["data"])
print(len(states), "Census rows;", len(power), "EIA rows")
```

```
52 Census rows; 62 EIA rows
```

```
# Given: the Census Bureau's crosswalk, a text file with "|" between columns; dtype=str keeps
 ↪  "08" as "08"
crosswalk_url = "https://www2.census.gov/geo/docs/reference/codes2020/national_state2020.txt"
codes = pd.read_csv(io.StringIO(requests.get(crosswalk_url, headers=HEADERS).text), sep="|",
 ↪  dtype=str)
codes.head(3)
```

```
  STATE STATEFP   STATENS STATE_NAME
0    AL      01  01779775    Alabama
1    AK      02  01785533     Alaska
2    AZ      04  01779777    Arizona
```

```
# Merge states with codes[["STATEFP", "STATE"]] on state = STATEFP (how="left"); then joined:
 ↪  merge with power on STATE = stateid,
#   how="outer", indicator=True. Print joined["_merge"].value_counts(), and the names of the rows
 ↪  on each side that didn't match
states = states.merge(codes[["STATEFP", "STATE"]], left_on="state", right_on="STATEFP",
 ↪  how="left")
joined = states.merge(power, left_on="STATE", right_on="stateid", how="outer", indicator=True)
print(joined["_merge"].value_counts())
print("Only Census:", list(joined.loc[joined["_merge"] == "left_only", "NAME"]))
print("Only EIA:", list(joined.loc[joined["_merge"] == "right_only", "stateDescription"]))
```

```
_merge
both          51
right_only    11
left_only      1
Name: count, dtype: int64
Only Census: ['Puerto Rico']
Only EIA: ['East North Central', 'East South Central', 'Middle Atlantic', 'Mountain', 'New
 ↪  England', 'Pacific Contiguous', 'Pacific Noncontiguous', 'South Atlantic', 'U.S. Total',
 ↪  'West North Central', 'West South Central']
```

### ⏱️ Microsprint (4 minutes, alone or in pairs)

Now the join can answer the question.

**Do:** keep the rows that matched in both; convert `revenue` and `customers` with `pd.to_numeric`; compute each state's average annual bill, `revenue * 1_000_000 / customers` (revenue is in millions of dollars), and the bill as a percent of `income`. The key shows the five largest shares, and Colorado's. **Share:** the state with the largest share, and Colorado's bill and share.

```
# Microsprint: both: joined's rows whose _merge is "both", copied; convert revenue and customers;
 ↪  add annual_bill and share
# Show the five largest shares (NAME, annual_bill, income, share), then Colorado's row
both = joined[joined["_merge"] == "both"].copy()
both["revenue"] = pd.to_numeric(both["revenue"])
both["customers"] = pd.to_numeric(both["customers"])
both["annual_bill"] = both["revenue"] * 1_000_000 / both["customers"]
both["share"] = both["annual_bill"] / both["income"] * 100
columns = ["NAME", "annual_bill", "income", "share"]
print(both.sort_values("share",
 ↪  ascending=False)[columns].head(5).round(1).to_string(index=False))
print(both.loc[both["STATE"] == "CO", columns].round(1).to_string(index=False))
```

```
          NAME  annual_bill  income  share
   Mississippi       1857.9 56447.0    3.3
       Alabama       2082.0 63999.0    3.3
 West Virginia       1857.1 59608.0    3.1
     Louisiana       1691.5 60756.0    2.8
South Carolina       1794.1 69324.0    2.6
    NAME  annual_bill  income  share
Colorado       1206.9 95470.0    1.3
```

### 💬 Discuss

- An inner join would have dropped the unmatched rows without a word. Which of them would you have missed, and which would have quietly counted states twice in a sum of the EIA's column?

- The EIA's numbers are for 2024; the ACS estimate pools 2020 to 2024. Is that close enough for this question? For which questions wouldn't it be?
- Why join on codes through a crosswalk, rather than on the states' names, which both tables have?

## Step 20 — What did we learn? (3 minutes)

In a few sentences, as comments:

- The week's four APIs each carried an idea: what can I ask for, let the server compute, let the server filter, follow the links. Which one saved you the most code, and where?
- Step 18 paged with `next`; week 10 paged with `continue`, and Wednesday's Step 14 with `offset`. Which did you trust most to stop at the right place, and why?
- How many rows of Step 19's join didn't match, and what would the share of income have looked like if you hadn't looked?
- Which of this week's numbers would be different if you ran this notebook next week, and which wouldn't? Why?

**Recommended Exercises ahead** (in the chapter's companion notebook, due Sunday):

- Step 1, load your keys without printing them: Step 0 and Step 7's `os.environ[...]`, and Step 0's Discuss on where keys leak.
- Step 2, an ACS variable for every Colorado county: Step 1's catalog for the variable, Step 2's catalog for what a county requires, and Step 6's wildcard, which asked for every place.
- Step 3, build the DataFrame, convert, sort, and print the extremes: Step 3's `pd.to_numeric`, and Step 6's `describe()` and `mask()` for the codes that aren't numbers.
- Step 4, a bar chart of the top 20 counties: Step 4's bar chart.
- Step 5, a related FRED series for Colorado over time: Steps 8 and 9's `fred_observations()`; find the series on FRED's website, as the chapter's tour does.
- Step 6, interpret: the Discuss questions after Steps 5, 6, 10, and 19.

```
# Your answers here, as comments
# 1. Letting the server filter saved the most. Wednesday's Step 13 got 25 rows, Colorado's homes'
 ↪  price by year, from a
#    route with every state and sector by month, quarter, and year; Step 19 got every state's
 ↪  revenue and customers,
#    62 rows, in one request. Following the links saved the most addresses: Steps 16 and 17 built
 ↪  none of their own.
# 2. next. Neguse's first page held all 88 of his 119th-Congress bills and a "next"; the second
 ↪  page had only a "prev",
#    so the loop stopped by itself. With the EIA's offset we kept count against "total" (9,289
 ↪  rows), and a mistake
#    in our arithmetic would have repeated or skipped rows without an error.
# 3. Twelve rows: Puerto Rico on the Census side, and on the EIA's ten regions and the U.S.
 ↪  total. Without looking,
#    a ranking would have put New England or the U.S. total among the states, and a sum would
 ↪  have
#    counted every state three times.
# 4. Congress's numbers will change: H.R. 7730's latest action (October 14), Neguse's 88 bills
 ↪  and Hurd's 27, and how
#    many are still in committee. The ACS 2020-2024 estimates and FRED's fixed windows won't; the
 ↪  EIA's 2025 figures
#    are preliminary, and its 2024 ones could be revised.
```

## Friday: from the lab to the textbook (15 minutes)

The last 15 minutes go to the textbook. Look back at what the lab turned up in chapter 11:

- **An output that has changed.** The chapter printed its outputs on October 25, 2026, while Congress was in session. Compare H.R. 7730's latest action (Step 16) and Joe Neguse's number of bills (Step 18) with the chapter's. Which outputs should the chapter fix to a date, and how?
- **A cell that broke.** Did a request fail? The lab's code assumes every request works; the chapter's calls `raise_for_status()`. A Congress.gov link sent without the `X-Api-Key` header fails with a 403: does "Common Issues to Debug" explain it well enough?
- **A sentence that confused.** "More Government APIs for Colorado" lists twelve more APIs. Try one: does its entry say enough to get started, and is its key requirement right?
- **A microsprint that would make a good exercise.** Following the `textVersions` link, or counting a member's bills still in committee, could be an Additional Exercise.

The revision framework says how to propose a change, and what each kind needs to be accepted.